

PERTEMUAN KE – 12

Session dan Cookie

A. TUJUAN

- Memahami konsep dan penggunaan session dan cookie

B. TEORI SINGKAT

PHP Cookies

Cookies adalah file teks yang disimpan pada komputer klien dan menetap. Cookies digunakan untuk pelacakan. PHP secara transparan mendukung HTTP cookie.

Ada tiga langkah yang terlibat dalam mengidentifikasi kembali pengguna:

- Sekrip Server mengirimkan satu set cookie ke browser. Sebagai Contoh nama, usia, atau nomor identifikasi dll
- Browser menyimpan informasi ini pada komputer lokal untuk penggunaan selanjutnya.
- Ketika browser mengirimkan permintaan ke server web maka mengirimkan cookie informasi ke server dan server menggunakan informasi untuk mengidentifikasi pengguna.

Bab ini akan mengajarkan cara mengatur cookies, bagaimana untuk mengaksesnya dan bagaimana untuk menghapusnya.

Anatomy Cookie:

Cookie biasanya ditetapkan dalam header HTTP (walaupun JavaScript juga dapat mengatur cookie langsung di browser). Sebuah sekrip PHP yang menetapkan cookie mungkin mengirim header yang terlihat seperti ini:

```
HTTP/1.1 200 OK
Date: Fri, 04 Feb 2000 21:03:38 GMT
Server: Apache/1.3.9 (UNIX) PHP/4.0b3
Set-Cookie: name=xyz; expires=Friday, 04-Feb-07 22:03:38 GMT;
           path=/; domain=domain.com
Connection: close
Content-Type: text/html
```

Seperti yang dilihat, header Set-Cookie berisi nilai pasangan name, date GMT, path dan domain. Name dan value akan diencode URL. Field `expires` berisi instruksi untuk browser untuk "forget" cookie setelah waktu tertentu dan tanggal.

Jika browser dikonfigurasi untuk menyimpan cookie, maka ia akan menyimpan informasi ini sampai tanggal kadaluwarsa. Jika pengguna menunjukkan browser pada halaman manapun yang sesuai dengan jalur dan domain dari cookie, ia akan mengirim ulang cookie untuk header server. The browser akan terlihat seperti ini:

```
GET / HTTP/1.0
Connection: Keep-Alive
User-Agent: Mozilla/4.6 (X11; I; Linux 2.2.6-15apmac ppc)
Host: zink.demon.co.uk:1126
Accept: image/gif, */*
Accept-Encoding: gzip
Accept-Language: en
Accept-Charset: iso-8859-1,*,utf-8
Cookie: name=xyz
```

Sebuah sekrip PHP maka akan memiliki akses ke cookie di lingkungan variabel `$_COOKIE` atau `$HTTP_COOKIE_VARS[]` yang memegang semua nama cookie dan nilai-nilai. Cookie dapat diakses menggunakan `$HTTP_COOKIE_VARS["name"]`.
Mengatur Cookies dengan PHP:

PHP tersedia `setcookie()` fungsi untuk mengatur cookie. Fungsi ini memerlukan upto enam argumen dan harus dipanggil sebelum `<html>` tag. Untuk setiap cookie fungsi ini harus dipanggil secara terpisah.

```
setcookie(name, value, expire, path, domain, security);
```

Berikut adalah detail dari semua argumen:

- `name` - menetapkan nama cookie dan disimpan dalam variabel lingkungan yang disebut `HTTP_COOKIE_VARS`. Variabel ini digunakan saat mengakses cookie.
- `value` - Ini menetapkan nilai variabel bernama dan konten yang Anda benar-benar ingin menyimpan.
- `expire` - menentukan waktu masa depan dalam detik sejak 00:00:00 GMT pada 1 Januari 1970. Setelah waktu ini kue akan menjadi tidak dapat diakses. Jika parameter ini tidak diatur maka kuki secara otomatis akan berakhir ketika Web Browser ditutup.
- `path` - menentukan direktori yang cookie berlaku. Sebuah maju karakter garis miring memungkinkan cookie untuk berlaku untuk semua direktori.
- `domain` - dapat digunakan untuk menentukan nama domain di domain yang sangat besar dan harus mengandung setidaknya dua periode menjadi valid. Semua cookie hanya berlaku untuk host dan domain yang menciptakan mereka.
- `security` - dapat diatur untuk 1 untuk menentukan bahwa cookie hanya boleh dikirim melalui aman menggunakan HTTPS dinyatakan diatur ke 0 yang berarti kue dapat dikirim melalui HTTP biasa.

PHP Sessions

Cara alternatif untuk membuat data yang dapat diakses di berbagai halaman seluruh situs Web adalah dengan menggunakan PHP Session.

Sesi menciptakan sebuah file di direktori sementara pada server di mana variabel sesi terdaftar dan nilai-nilai mereka disimpan. Data ini akan tersedia untuk semua halaman di situs selama kunjungan itu.

Lokasi file sementara ditentukan oleh pengaturan pada file `php.ini` disebut `session.save_path`. Bore menggunakan variabel sesi pastikan Anda memiliki setup jalan ini.

Ketika sesi dimulai hal-hal berikut terjadi:

- PHP pertama menciptakan identifikasi unik untuk sesi khusus yang merupakan string acak dari 32 angka heksadesimal seperti `3c7foj34c3jj973hjkop2fc937e3443`.
- Cookie disebut `PHPSESSID` secara otomatis dikirim ke komputer pengguna untuk menyimpan tali identifikasi sesi yang unik.
- Sebuah file secara otomatis dibuat pada server dalam direktori sementara ditunjuk dan menyandang nama pengenalan unik diawali oleh `sess_` yaitu `sess_3c7foj34c3jj973hjkop2fc937e3443`.

Ketika sekrip PHP ingin mengambil nilai dari variabel sesi, PHP secara otomatis mendapatkan unik identifier sesi string dari cookie `PHPSESSID` dan kemudian terlihat dalam direktori sementara untuk file yang mengandung nama itu dan validasi dapat dilakukan dengan membandingkan kedua nilai.

Sesi berakhir ketika pengguna kehilangan browser atau setelah meninggalkan situs, server akan mengakhiri sesi setelah jangka waktu tertentu, biasanya 30 menit durasi.

Memulai PHP Session:

Sesi PHP mudah dimulai dengan membuat panggilan ke `session_start()` function. This berfungsi cek pertama jika sesi sudah dimulai dan jika tidak dimulai maka mulai satu. Disarankan untuk menempatkan panggilan untuk `session_start()` di awal halaman.

Variabel sesi yang disimpan dalam array asosiatif disebut `$_SESSION[]`. Variabel ini dapat diakses selama masa sesi.

C. PRAKTIK

Praktik 1

Contoh berikut akan membuat nama dua cookies dan usia cookies tersebut akan berakhir setelah satu jam.

```
<?php
    setcookie("nama", "John Watkin", time() + 3600, "/", "", 0);
    setcookie("usia", "36", time() + 3600, "/", "", 0);
?>
<html>
<head>
<title> Pengaturan Cookies dengan PHP </title>
</head>
<body>
<?php echo "Set Cookies"?>
</body>
</html>
```

Praktik 2

Mengakses Cookies dengan PHP

PHP menyediakan banyak cara untuk mengakses cookies. Simplest cara adalah dengan menggunakan baik \$_COOKIE atau \$HTTP_COOKIE_VARS variabel. Contoh berikut akan mengakses semua diatur dalam contoh di atas kue. sess_3c7foj34c3jj973hjkop2fc937e3443

```
<html>
<head>
<title> Mengakses Cookies dengan PHP </title>
</head>
<body>
<?php
echo $_COOKIE["nama"]. "<br>";
/* Setara dengan */
echo $HTTP_COOKIE_VARS["nama"]. "<br>";

echo $_COOKIE["usia"]. "<br>";
/* Setara dengan */
echo $HTTP_COOKIE_VARS["usia"]. "<br>";
?>
</body>
</html>
```

Praktik 3

Anda dapat menggunakan isset() fungsi untuk memeriksa apakah ada cookie yang dibuat atau tidak.

```
<html>
<head>
<title> Mengakses Cookies dengan PHP </title>
</head>
<body>
<?php
    if(isset($_COOKIE["nama"]))
        echo "Selamat datang ". $_COOKIE["nama"]. "<br>";
    else
        echo "Maaf ... Tidak diakui". "<br>";
?>
</body>
</html>
```

Praktik 4

Menghapus Cookie dengan PHP Contoh berikut mulai sesi kemudian mendaftarkan variabel yang disebut counter yang bertambah setiap kali halaman dikunjungi selama sesi.

Secara resmi, untuk menghapus cookie harus menghubungi `setcookie()` dengan argumen nama saja tetapi hal ini tidak selalu bekerja dengan baik, dengan demikian, tidak boleh diandalkan.

Cara paling aman untuk mengatur cookie dengan tanggal yang sudah kadaluarsa:

```
<?php
    setcookie("nama", "", time() - 60, "/", "", 0);
    setcookie("usia", "", time() - 60, "/", "", 0);
?>
<html>
<head>
<title> Menghapus Cookies dengan PHP </title>
</head>
<body>
<?php echo "Dihapus Cookies"?>
</body>
</html>
```

Praktik 5

Contoh berikut mulai sesi kemudian mendaftarkan variabel yang disebut counter yang bertambah setiap kali halaman dikunjungi selama sesi.

Memanfaatkan `isset()` fungsi untuk memeriksa apakah variabel sesi sudah diatur atau tidak.

Masukan kode ini dalam file `test.php` dan load file ini berkali-kali untuk melihat hasilnya:

```
<?php
    session_start();
    if(isset($_SESSION['counter']))
    {
        $_SESSION['counter']+= 1;
    }
    else
    {
        $_SESSION['counter']= 1;
    }
    $msg = "Anda telah mengunjungi halaman ini ". $_SESSION['counter'];
    $msg .= " di sesi ini.";
?>
<html>
<head>
<title> Menyiapkan sesi PHP </title>
</head>
<body>
<?php echo($msg); ?>
</body>
</html>
```

Praktik 6

Menghancurkan PHP Session:

Sesi PHP dapat dihancurkan oleh `session_destroy()` function. Fungsi ini tidak membutuhkan argumen dan satu panggilan dapat menghancurkan semua variabel sesi. Jika Anda ingin menghancurkan sebuah variabel sesi tunggal maka Anda dapat menggunakan `unset()` fungsi unset variabel sesi.

Berikut adalah contoh unset variabel tunggal:

```
<?php
```

```

        session_start();
        unset($_SESSION['counter']);
    ?>

```

Berikut adalah panggilan yang akan menghancurkan semua variabel sesi:

```

<?php
    session_start();
    session_destroy();
?>

```

Praktik 7

Mengaktifkan Auto Sesi:

Fungsi `session_start()` tidak perlu dipanggil untuk memulai sesi ketika pengguna mengunjungi situs jika diatur variabel `session.auto_start` 1 di file `php.ini`.

Praktik 8

Sesi tanpa cookie:

Mungkin ada kasus ketika pengguna tidak memungkinkan untuk menyimpan cookie pada mesin mereka. Jadi ada metode lain untuk mengirim ID sesi ke browser.

Atau, Anda dapat menggunakan SID konstan yang didefinisikan jika sesi dimulai. Jika klien tidak mengirim cookie sesi yang tepat (bentuk `session_name = session_id`). Jika tidak, klien mengembangkan menjadi string kosong. Dengan demikian, dapat ditanamkan tanpa syarat ke URL.

Contoh berikut menunjukkan cara mendaftar variabel, dan bagaimana menghubungkan dengan benar ke halaman lain menggunakan SID.

```

<?php
    session_start();

    if(isset($_SESSION['counter'])) {
        $_SESSION['counter'] = 1;
    } Else {
        $_SESSION['counter']++;
    }
    $msg = "Anda telah mengunjungi halaman ini ". $_SESSION['counter'];
    $msg .= " di sesi ini.<br>";
    echo($msg);
    echo htmlspecialchars(SID);
?>
<p>
Untuk melanjutkan klik link berikut <br>
<a href="nextpage.php?".<?php echo htmlspecialchars(SID); ?>."
>nextpage.php
</p>

```

D. LATIHAN

- Latihan diberikan oleh dosen pengampu pada saat praktikum.
- Dikerjakan di laboratorium pada jam praktikum.

E. TUGAS

- Tugas diberikan oleh dosen pengampu pada akhir praktikum.
- Dikerjakan di rumah dan dilampirkan pada laporan.